Neural Networks and Deep Learning

Chapter based on notes by Dr Kevin Webster

In this chapter we introduce neural network architectures, and with them the idea of so-called *deep learning* by stacking together several layers of computing nodes.

1. The mathematical neuron

Early research into neural networks was based on models of learning in the brain using mathematical (or artificial) neurons as fundamental building blocks. These are simple information-processing units, originally meant as models of neurons in the brain, that receive a set of inputs, which are weighted and summed before being passed through an activation function to give the output y:

$$y = \sigma\left(\sum_i w_i x_i + b\right) \tag{7.1}$$

Here the inputs to the neuron are denoted by x_i, the weights by w_i, and the activation function by σ, while b stands for a bias added to the weighted sum of neurons.

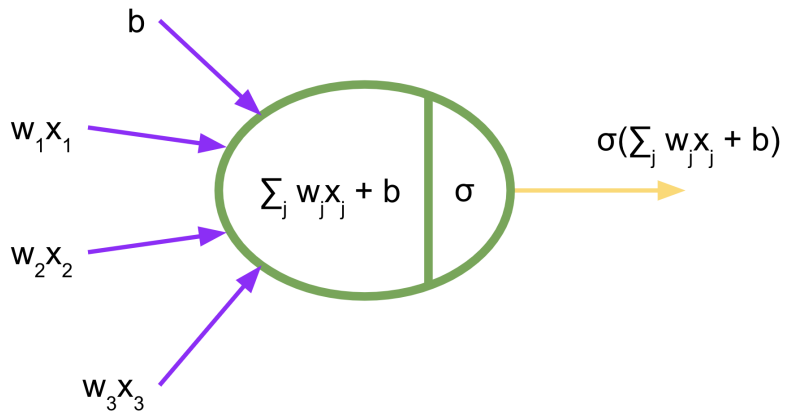


Figure 7.1. Sketch of a mathematical neuron.

The weights and bias are the parameters that need to be tuned (or *learnt*) for the given prediction task in the training of networks of such basic computing units (neural networks). This step is made, as we have seen for other methods, by optimisation of a properly defined loss function through gradient descent algorithms, which motivated research on smooth activation functions (see aside).

> *Quick Aside.* The first artificial neuron was developed by McCulloch and Pitts [9], which used a simple threshold activation function (step function) only on binary inputs, and produced a binary output. Later, Rosenblatt [14] developed the *perceptron*, which also used a step function threshold for binary classification (but with more general weights and inputs), and importantly also introduced a learning algorithm for the weights. The perceptron learning algorithm is guaranteed to converge for linearly separable data. However, the limitations of linear models was largely responsible for the decline in interest in neural networks until its revival in the 1980s. The second wave of interest in neural networks in the 80s was driven in large part by the connectionist movement (see e.g. Rumelhart et al. [15]), which focused on the concept of intelligent behaviour arising out of many simple computations composed together, with knowledge being distributed across many units. Smooth activation functions were increasingly studied, as they allowed gradient-based methods to be used in the optimisation of model parameters. For a more complete historical account on the research into neural networks, we refer to the book Goodfellow, Bengio, Courville *Deep learning* (2016), chapter 2, section 1.2.

1.1. Activation functions. A typical example of a smooth activation function is the sigmoid (or logistic function, that we have seen already in Chapter 3):

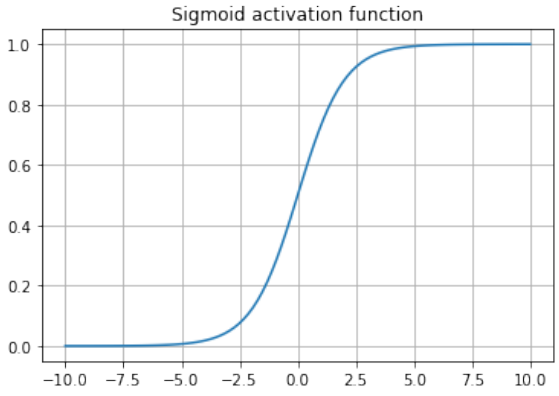


Figure 7.2. Sigmoid activation function.

Indeed, both linear regression and logistic regression can be viewed as basic artificial neuron models, with linear activation function and sigmoid activation function, respectively.

Other activation functions that later started to be commonly used in deep learning models are the ReLU (rectified linear unit), tanh, ELU (exponential linear unit, Clevert et al. 2016 [1]), SELU (scaled exponential linear unit, Klambauer et al. 2017 [6]), softplus, swish (Ramachandran et al. 2018 [13]), which can be seen in Figure 7.3.

2. Multilayer Perceptron

The simplest type of deep learning model is the **Multi-Layer Perceptron** (MLP), also simply known as a **feedforward network**. This type of neural network can be viewed as

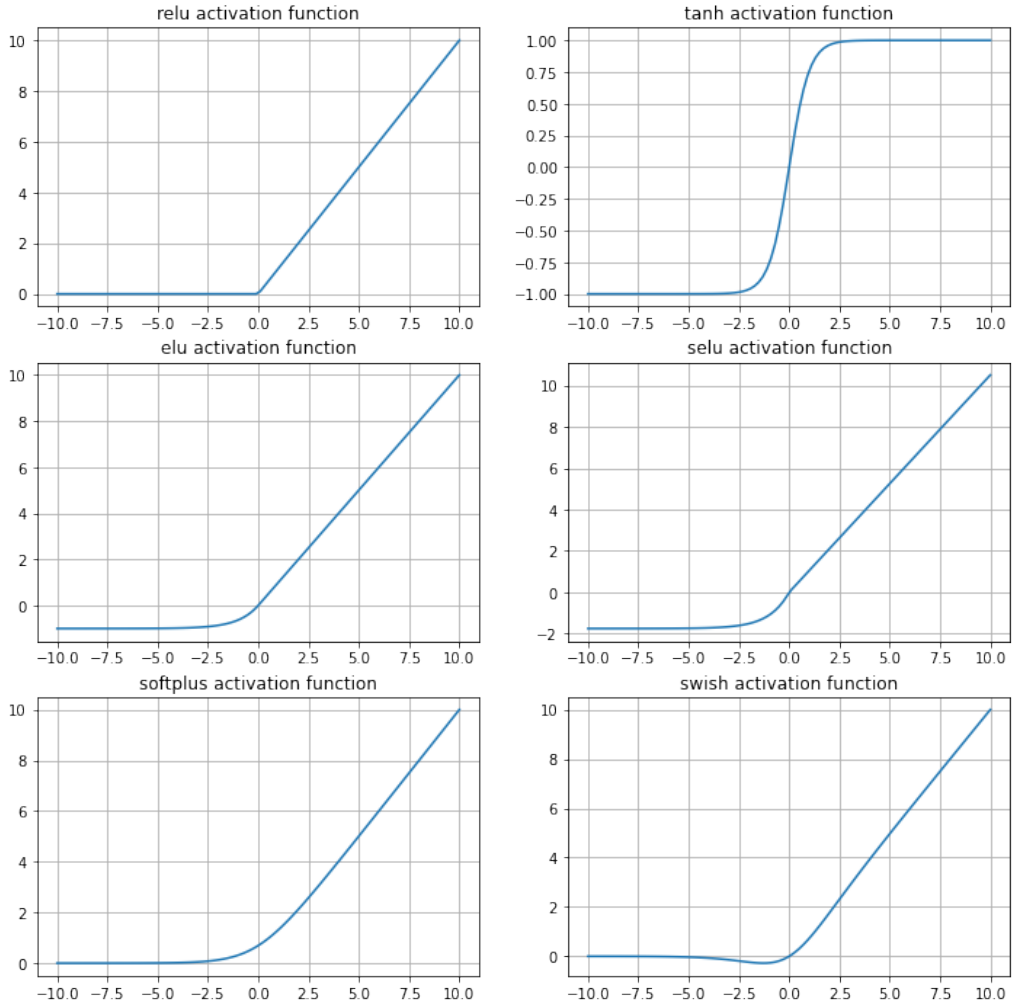


Figure 7.3. A variety of activation functions.

an architecture consisting of layers of mathematical neurons, linked together in a directed acyclic graph, i.e., by a graph where the information is processed uni-directionally from the input to the output.

2.1. MLP architecture. A key property of deep learning models is the fact that they are *compositional* instead of *additive*. By this we mean that deep learning models increase complexity by composing multiple simple functions φ_l together, to evaluate on some input data $\mathbf{x} = (x_1, \cdots, x_p)$:

$$f(\mathbf{x}) = \varphi_{N_L}(\varphi_{N_L-1}(\ldots \varphi_2(\varphi_1(\mathbf{x}))\ldots))$$

up to a certain N_L, which stands for the number of 'hidden' (i.e., internal) layers. The functions φ_l are defined to be affine transformations followed by an element-wise activation function.

Let's start with a simple example of such an $f(\mathbf{x})$, an MLP with a single hidden layer (i.e., $N_L = 1$):

$$h_j^{(1)} = \sigma\left(\sum_{i=1}^{p} w_{ji}^{(0)} x_i + b_j^{(0)}\right), \qquad j = 1, \ldots, n_h, \tag{7.2}$$

$$\hat{y} = \sigma_{out}\left(\sum_{j=1}^{n_h} w_j^{(1)} h_j^{(1)} + b^{(1)}\right). \tag{7.3}$$

In the above, $n_h \in \mathbb{N}$ is the number of hidden units of this (single-hidden-layer) network, $\sigma, \sigma_{out} : \mathbb{R} \mapsto \mathbb{R}$ are activation functions, $w_{ji}^{(0)} \in \mathbb{R}$ and $w_{ji}^{(1)} \in \mathbb{R}$ are weights, and $b_j^{(0)} \in \mathbb{R}$ and $b^{(1)} \in \mathbb{R}$ are biases.

This construction is summarised in Figure 7.4.

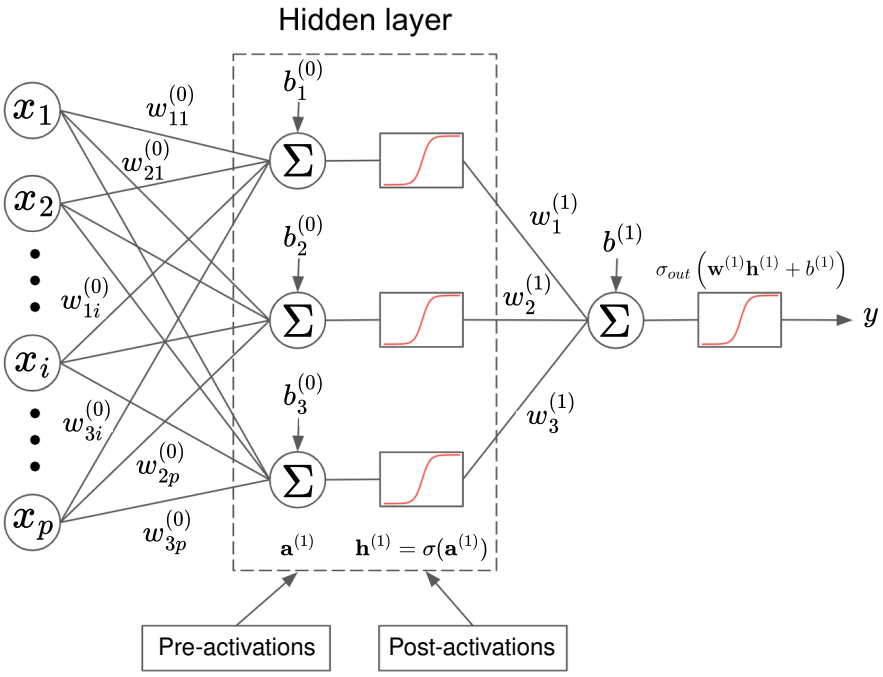


Figure 7.4. Multilayer perceptron with a single hidden layer consisting of $n_h = 3$ neurons.

We will usually write equations 7.2 and 7.3 in the more concise form:

$$\mathbf{h}^{(1)} = \sigma\left(\mathbf{W}^{(0)}\mathbf{x} + \mathbf{b}^{(0)}\right), \tag{7.4}$$

$$\hat{y} = \sigma_{out}\left(\mathbf{w}^{(1)}\mathbf{h}^{(1)} + b^{(1)}\right), \tag{7.5}$$

where $\mathbf{x} \in \mathbb{R}^p$, $\mathbf{W}^{(0)} \in \mathbb{R}^{n_h \times p}$, $\mathbf{b}^{(0)} \in \mathbb{R}^{n_h}$, $\mathbf{h}^{(1)} \in \mathbb{R}^{n_h}$, $\mathbf{w}^{(1)} \in \mathbb{R}^{1 \times n_h}$, $b^{(1)} \in \mathbb{R}$ and the activation functions $\sigma, \sigma_{out} : \mathbb{R} \mapsto \mathbb{R}$ are applied element-wise in the above. This hidden layer is a type of neural network layer that is often referred to as a **dense** or **fully connected** layer, i.e., with all-to-all connections (all input features are connected to all hidden units and in turn all hidden units to the output unit by the weights).

Key message: Generalising this construction to multiple hidden layers N_L, the general MLP architecture is:

$$\mathbf{h}^{(0)} := \mathbf{x}, \tag{7.6}$$

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l-1)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l-1)}\right), \qquad l = 1, \ldots, N_L, \tag{7.7}$$

$$\hat{y} = \sigma_{out}\left(\mathbf{w}^{(N_L)}\mathbf{h}^{(N_L)} + b^{(N_L)}\right), \tag{7.8}$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{n_{l+1} \times n_l}$, $\mathbf{b}^{(l)} \in \mathbb{R}^{n_{l+1}}$, $\mathbf{h}^{(l)} \in \mathbb{R}^{n_l}$, and we have set $n_0 := p$, and n_l is the number of units in the l-th hidden layer. We define the $\mathbf{h}^{(l)}$ the **post-activations** (or frequently, just **activations**):

$$\mathbf{h}^{(l)} = \sigma(\mathbf{a}^{(l)}), \tag{7.9}$$

where:

$$\mathbf{a}^{(l)} = \mathbf{W}^{(l-1)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l-1)} \tag{7.10}$$

gives the **pre-activations**, with $l = 1, \ldots, N_L$.

Figure 7.5 shows the MLP architecture with two hidden layers.

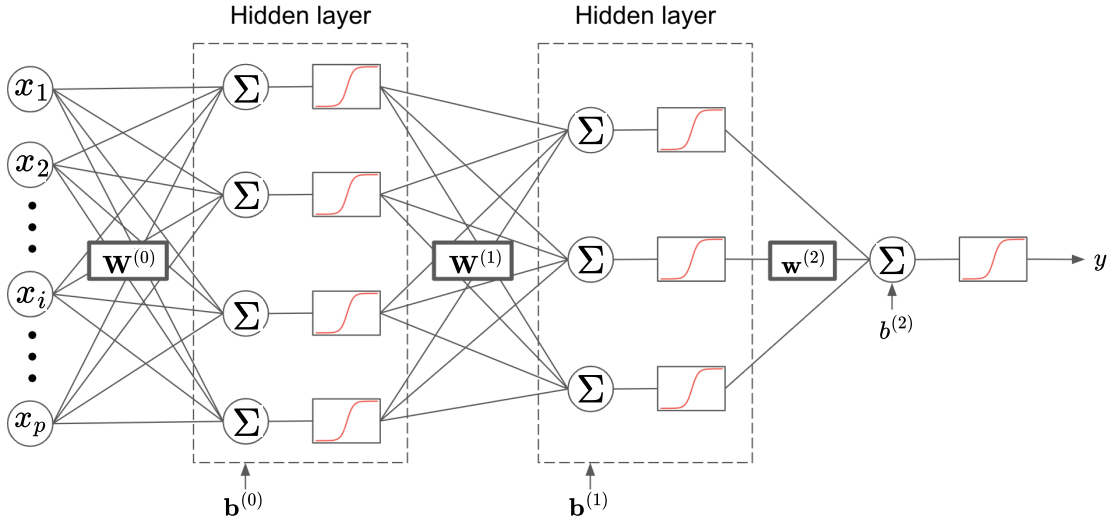


Figure 7.5. Multilayer perceptron with $N_L = 2$ hidden layers.

The hidden layers inside a deep network can be viewed as *learned feature extractors*. The weights of the network learn to encode the data in such a way as to represent progressively more complex or abstract features of the data that are useful for solving the problem task at hand. **This hierarchy of representations is a core property of the expressive power of deep learning** - see the review [7] which is also proposed in the "additional reading" folder.

2.2. Output layers. One of the strengths of deep learning models is their applicability to a wide range of data set types and problem tasks. In equation 7.8 we have considered a single unit output y, which is produced by passing the pre-activation $a^{(N_L+1)} := \mathbf{w}^{(N_L)}\mathbf{h}^{(N_L)} + b^{(N_L)}$ through the activation function σ_{out}.

Note how linear regression (chapter 1) and logistic regression (chapter 3) can both be viewed as a neural network without a hidden layer. In this case, if σ_{out} is be the identity (or linear) activation, then we are left with a simple linear regression model. Likewise, if σ_{out} is the sigmoid function, then we have the logistic regression model.

The architecture can also be easily modified to output multiple target variables $\hat{\mathbf{y}}$ by replacing Equation 7.8 with:

$$\hat{\mathbf{y}} = \sigma_{out}\left(\mathbf{W}^{(N_L)}\mathbf{h}^{(N_L)} + \mathbf{b}^{(N_L)}\right), \tag{7.11}$$

where $\hat{\mathbf{y}}$ consists of $n_{N_L+1} > 1$ elements, as can be seen in Figure 7.6 for $n_{N_L+1} = 3$.

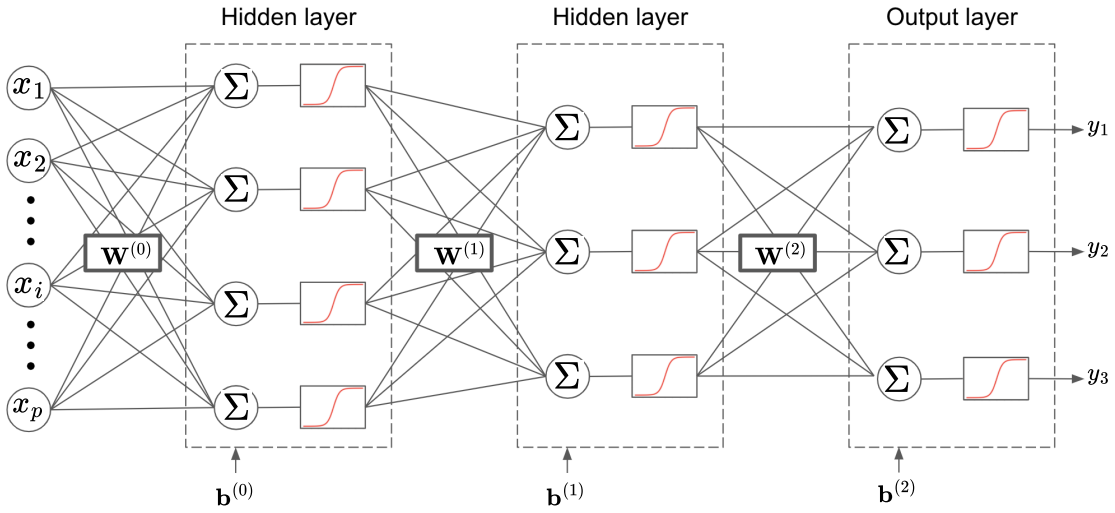


Figure 7.6. Multilayer perceptron with multiple outputs.

Moreover, the activation functions in the output layer can be chosen according to the requirements of the target variables. For example, if the network should output an estimate for a standard deviation parameter, then we will want to constrain the output to be positive. This can be achieved by passing the pre-activation through a softplus or exponential activation function, for example. It is common for a sigmoid activation to be used where the output should be interpreted as a probability (as in logistic regression). More generally, for target variables that should be constrained to an interval, then a sigmoid or tanh activation can be used followed by a suitable rescaling. Different activation functions could be applied to different units in the output layer, if appropriate.

A common output layer used for multiclass classification models is the softmax. The softmax layer outputs a normalised array, which can be interpreted as a probability vector specifying a categorical distribution. We have pre-activations:

$$\mathbf{a}^{(N_L+1)} := \mathbf{W}^{(N_L)}\mathbf{h}^{(N_L)} + \mathbf{b}^{(N_L)}$$

with $\mathbf{W}^{(N_L)} \in \mathbb{R}^{Q \times n_{N_L}}$, $\mathbf{b}^{(N_L)} \in \mathbb{R}^{Q}$ where Q is the number of classes (i.e., the number of output units). The softmax function is given by:

$$\hat{\mathbf{y}}_q := \text{softmax}(\mathbf{a}^{(N_L+1)})_q = \frac{\exp(a_q)}{\sum_{q'=1}^{Q}\exp(a_{q'})}, \qquad q = 1, \cdots, Q$$

where each element takes values between 0 and 1 and the vector is normalised over the Q classes by construction. As such, the soft-max is suitable to build a probabilistic classifier (soft classification). Note that the softmax function operates on all pre-activations in the output layer, in contrast to the usual element-wise application of most activation functions - see Figure 7.7 for a schematic.

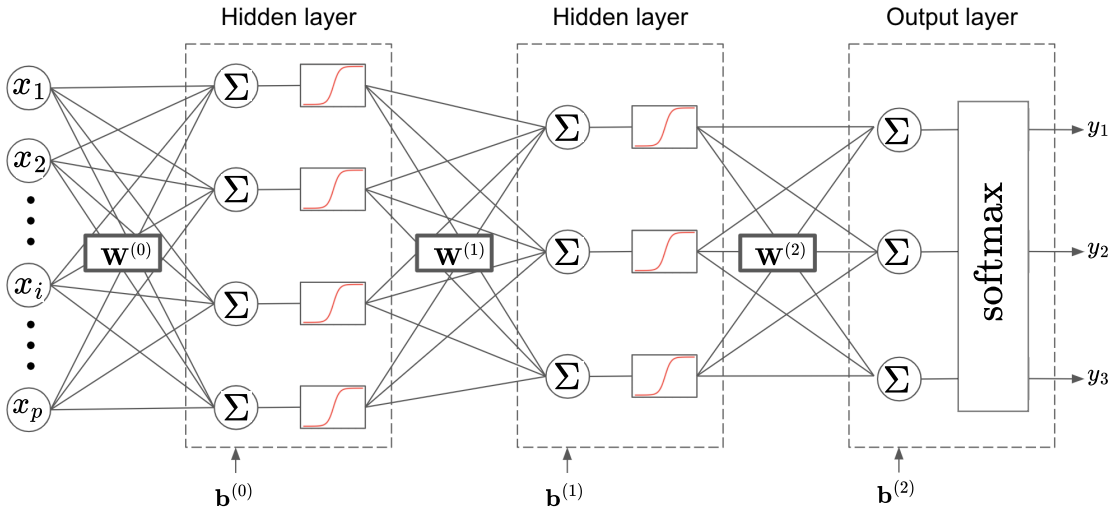


Figure 7.7. Multilayer perceptron with a softmax output layer for soft classification.

3. Neural Network training: error backpropagation

Let us consider the usual setup, where the data available for a supervised learning task is:

$$\{\mathbf{x}^{(i)}, y^{(i)}\} \qquad i = 1, \ldots, N, \qquad \mathbf{x}^{(i)} \in \mathbb{R}^p \qquad y^{(i)} \in Y$$

and where the target space Y can be either continuous (for a regression task) or categorical (for a classification task). Suppose we have constructed our neural network model, which we represent as the function $f_{\boldsymbol{\theta}} : \mathbb{R}^p \mapsto Y$, that depends on a set of parameters $\boldsymbol{\theta}$ (containing weights and biases) to be learnt. Suppose also that we have defined a suitable loss function for training:

$$L(\boldsymbol{\theta}; \mathcal{S}) := \frac{1}{|\mathcal{S}|} \sum_{\mathbf{x}^{(i)}, y^{(i)} \in \mathcal{S}} L(y^{(i)}, f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})), \tag{7.12}$$

where $\mathcal{S} = \{\mathbf{x}^{(i)}, y^{(i)}\}$ stands for the training set, with $i = 1, \ldots, N^{\text{training}}$, and where $L(y^{(i)}, f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ is the per-example loss. Also **in neural network architectures the parameters $\boldsymbol{\theta}$ (i.e., weights and biases) are found by solving an optimisation problem, that is, by minimising the loss function** (7.12).

To this end, we appeal to gradient-based optimisation methods, and in particular to the minibatch stochastic gradient descent (minibatch SGD) algorithm introduced in Chapter 6. Schematically, gradient-based neural network optimisation can be seen as iterating over the following two main steps:

(1) Computation of the (stochastic) gradient of the loss function with respect to the model parameters;

(2) Use of the computed gradient to update the parameters.

The parameter update rule (for step 2) according to minibatch SGD for a neural network model $f_{\boldsymbol{\theta}} : \mathbb{R}^p \mapsto Y$ is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t; \mathcal{S}_m) \tag{7.13}$$

In the above equation, the minibatch loss at iteration t $L(\boldsymbol{\theta}_t; \mathcal{S}_m)$ is calculated on a randomly drawn sample of data points from the training set, as we have seen in Chapter

6:

$$L(\boldsymbol{\theta}_t; \mathcal{S}_m) = \frac{1}{|\mathcal{S}_m|} \sum_{\mathbf{x}^{(i)}, y^{(i)} \in \mathcal{S}_m} L(y^{(i)}, f_{\boldsymbol{\theta}_t}(\mathbf{x}^{(i)})), \tag{7.14}$$

where $\mathcal{S}_m$ is the randomly sampled minibatch, $|\mathcal{S}_m| \ll |\mathcal{S}|$ is its size. We will denote by $L_i : Y \times Y \mapsto \mathbb{R}$, given by $L_i := L(y^{(i)}, f_{\boldsymbol{\theta}_t}(\mathbf{x}^{(i)}))$, the per-example loss. For a suitably chosen η_t, minibatch SGD converges to a minimum in the parameter space - but typically only a *local* minimum, since the loss parametrised by neural networks is highly non-convex.

The update 7.13 requires the computation of the gradient of the loss function with respect to all of the model parameters, evaluated at the current parameter settings $\boldsymbol{\theta}_t$.

The algorithm for computing these derivatives in an efficient manner is known as *backpropagation*, and is based on the chain rule of differentiation. In this section, we will derive the main steps of the backpropagation algorithm for the MLP.

> *Quick Aside.* The backpropagation algorithm was popularised for use in neural network optimisation in Rumelhart et al. 1986b [16] and Rumelhart et al. 1986c [17], although the technique dates back earlier, see e.g. Werbos [19] which includes Paul Werbos' 1974 dissertation.

First recall the layer transformations of the MLP (Equations 7.6 - 7.8), potentially with a multi-dimensional output as in Equation 7.11, as well as the definitions of pre-activations and post-activations (Equations 7.10-7.9).

We will consider the gradient of the loss computed on a single data example $\nabla_{\boldsymbol{\theta}} L_i$, given the sum 7.14. We first compute the **forward pass** 7.6 - 7.8 and store the preactivations $\mathbf{a}^{(l)}$ and post-activations $\mathbf{h}^{(l)}$.

Consider the derivative of L_i with respect to $w_{kq}^{(l)}$ and $b_k^{(l)}$. We have:

$$\frac{\partial L_i}{\partial w_{kq}^{(l)}} = \frac{\partial L_i}{\partial a_k^{(l+1)}} \frac{\partial a_k^{(l+1)}}{\partial w_{kq}^{(l)}} = \frac{\partial L_i}{\partial a_k^{(l+1)}} h_q^{(l)}, \tag{7.15}$$

where the second step follows from 7.10. Similarly,

$$\frac{\partial L_i}{\partial b_k^{(l)}} = \frac{\partial L_i}{\partial a_k^{(l+1)}} \frac{\partial a_k^{(l+1)}}{\partial b_k^{(l)}} = \frac{\partial L_i}{\partial a_k^{(l+1)}}. \tag{7.16}$$

We introduce the notation $\delta_k^{(l)} := \frac{\partial L_i}{\partial a_k^{(l)}}$, called the **error**. We then write:

$$\frac{\partial L_i}{\partial w_{kq}^{(l)}} = \delta_k^{(l+1)} h_q^{(l)}, \tag{7.17}$$

$$\frac{\partial L_i}{\partial b_k^{(l)}} = \delta_k^{(l+1)}. \tag{7.18}$$

We therefore need to compute the quantity $\delta_k^{(l+1)}$ for each hidden and output unit in the network. Again using the chain rule, we have:

$$\delta_k^{(l)} \equiv \frac{\partial L_i}{\partial a_k^{(l)}} = \sum_{j=1}^{n_{l+1}} \frac{\partial L_i}{\partial a_j^{(l+1)}} \frac{\partial a_j^{(l+1)}}{\partial a_k^{(l)}} = \sum_{j=1}^{n_{l+1}} \delta_j^{(l+1)} \frac{\partial a_j^{(l+1)}}{\partial a_k^{(l)}} \tag{7.19}$$

Combining 7.10 and 7.9 we see that:

$$a_j^{(l+1)} = \sum_{n=1}^{n_l} w_{jn}^{(l)} \sigma(a_n^{(l)}) + b_j^{(l)} \tag{7.20}$$

$$\frac{\partial a_j^{(l+1)}}{\partial a_k^{(l)}} = w_{jk}^{(l)} \sigma'(a_k^{(l)}) \tag{7.21}$$

where σ' is the derivative of the activation. From the above equations and 7.19 one has:

$$\delta_k^{(l)} \equiv \frac{\partial L_i}{\partial a_k^{(l)}} = \sum_{j=1}^{n_{l+1}} \delta_j^{(l+1)} \frac{\partial a_j^{(l+1)}}{\partial a_k^{(l)}} = \sigma'(a_k^{(l)}) \sum_{j=1}^{n_{l+1}} w_{jk}^{(l)} \delta_j^{(l+1)} \tag{7.22}$$

Equation 7.22, giving a rule determining the errors in the layer l as a function of the ones in the layer $l+1$, describes the *backpropagation* of errors through the network.

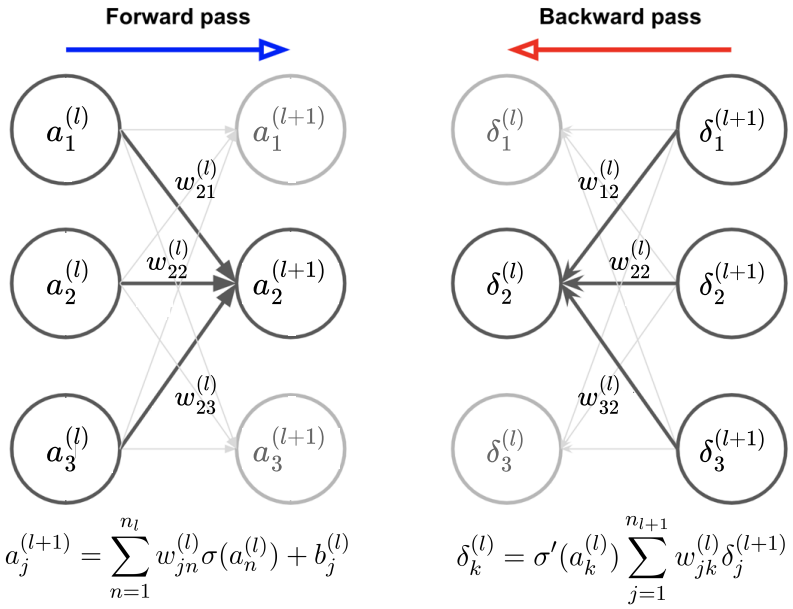


Figure 7.8. Forward and backward passes.

Key message: The backpropagation algorithm works as follows:

(1) Propagate the signal forwards by passing an input vector $\mathbf{x}^{(i)}$ through the network and computing all pre-activations and post-activations using:

$$\mathbf{a}^{(l)} = \mathbf{W}^{(l-1)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l-1)} \qquad l = 1, \cdots, N_L.$$

(2) Evaluate $\delta^{(N_L+1)} = \frac{\partial L_i}{\partial \mathbf{a}^{(N_L+1)}}$ for the output neurons (i.e., at layer N_L+1).

(3) Backpropagate the errors to compute $\delta^{(l)}$ for each hidden unit using Equation (7.22), whose compact form reads:

$$\delta^{(l)} = \sigma'(\mathbf{a}^{(l)}) (\mathbf{W}^{(l)})^T \delta^{(l+1)},$$

where $\sigma'(\mathbf{a}^{(l)}) = \text{diag}([\sigma'(a_k^{(l)})]_{k=1}^{n_l})$ and the activation functions are applied element-wise.

(4) Obtain the derivatives of L_i with respect to the weights and biases using: $\frac{\partial L_i}{\partial w_{kq}^{(l)}} = \delta_k^{(l+1)} h_q^{(l)}$, $\quad \frac{\partial L_i}{\partial b_k^{(l)}} = \delta_k^{(l+1)}$.

The derivatives with respect to the weights and biases are then used in the gradient-descent algorithm to train the network.

4. Neural Network training: Optimisers

Recall the two main steps to training neural networks:

(1) Computation of the (stochastic) gradient of the loss function with respect to the model parameters;

(2) Use of the computed gradient to update the parameters.

Now that we have seen how gradients of the loss with respect to the parameters can be efficiently computed using the backpropagation algorithm (step 1), we will take a look at several popular gradient-based optimisation algorithms used in deep learning (step 2).

We have already seen that minibatch SGD can be applied to optimise neural network parameters, and its advantages (SGD reduces redundancy in the gradient computation, and is faster than standard gradient descent). However some challenges remain:

- Convergence can still be very slow with minibatch SGD;
- Setting the learning rate correctly can be difficult, involving trial and error;
- Different weights might operate on different scales, thus requiring different rates of learning.

Several optimisation algorithms have been proposed to help cure these problems.

4.1. Momentum. One common tweak to accelerate the slow convergence of minibatch SGD is to add momentum (Qian 1999 [12]):

$$\begin{aligned} \mathbf{g}_t &:= \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t; \mathcal{S}_m), \\ \mathbf{v}_{t+1} &= \beta \mathbf{v}_t + \eta \mathbf{g}_t \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \mathbf{v}_{t+1}, \end{aligned}$$

where $\beta \geq 0$ is the momentum term, and as before, $\eta > 0$ is the learning rate. When $\beta = 0$ then we recover plain minibatch SGD, but with $\beta > 0$ (a typical value is around 0.9), this gives the gradient a short term memory which often accelerates convergence.

4.2. Nesterov momentum. A common variant of momentum is to use Nesterov momentum (Nesterov 1983 [10]), which computes the gradient correction after the accumulated gradient, instead of before:

$$\begin{aligned} \mathbf{g}_t &= \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t - \beta \mathbf{v}_t; \mathcal{S}_m), \\ \mathbf{v}_{t+1} &= \beta \mathbf{v}_t + \eta \mathbf{g}_t \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \mathbf{v}_{t+1}. \end{aligned}$$

The accumulated gradient approximates the next value of the parameters, and so evaluating the gradient $\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t - \beta \mathbf{v}_t; \mathcal{S}_m)$ gives the optimiser a capability of 'look-ahead'.

4.3. Adagrad. The Adagrad optimiser (Duchi et al. 2011 [3]) adapts the learning rate for each parameter, to account for different weights learning on different scales. Parameters that receive a gradient less frequently have larger updates, making Adagrad well suited to sparse data, where most of the features are zero in the data. It is used, for example, in Pennington et al. 2014 [11] to train GloVe word embedding vectors.

The update rule is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{G_t + \varepsilon}} \odot \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t; \mathcal{S}_m),$$

where G_t is a diagonal matrix where the diagonal element $(G_t)_{ii}$ is the sum of squares of gradients with respect to θ_i up to the iteration t. In the above, the division and square root operations are performed element-wise, and $\odot$ is the Hadamard product.

Note that the resulting learning rates per parameter are monotonically decreasing, and eventually the algorithm effectively stops learning.

4.4. RMSprop. RMSprop is an optimisation method that aims to resolve the vanishing learning rates of Adagrad (it appeared in Geoff Hinton's Coursera course[1] in lecture 6e). It uses a decaying average of past squared gradients. Denoting the gradient by $\mathbf{g}$, the update rule is:

$$\begin{aligned}
\mathbb{E}[\mathbf{g}^2]_t &= \rho \mathbb{E}[\mathbf{g}^2]_{t-1} + (1-\rho)(\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t; \mathcal{S}_m))^2, \\
\boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\mathbb{E}[\mathbf{g}^2]_t + \varepsilon}} \odot \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t; \mathcal{S}_m).
\end{aligned}$$

As before, the division and square root are performed element-wise, and $\odot$ is the Hadamard product. The ρ term is typically set similar to momentum, around 0.9.

4.5. Adam. The Adam optimiser (Kingma 2015 [5]) is a very popular optimisation algorithm, that also computes adaptive learning rates per parameter. It estimates first and second moments of the gradients, and the name stands for Adaptive moment estimation.

Again, denoting the gradient by $\mathbf{g}$, the update rule is:

$$\begin{aligned}
\mathbb{E}[\mathbf{g}]_t &= \beta_1 \mathbb{E}[\mathbf{g}]_{t-1} + (1-\beta_1) \nabla_\theta L(\theta_t; \mathcal{D}_m), \\
\mathbb{E}[\mathbf{g}^2]_t &= \beta_2 \mathbb{E}[\mathbf{g}^2]_{t-1} + (1-\beta_2)(\nabla_\theta L(\theta_t; \mathcal{D}_m))^2, \\
\mathbf{m}_t &= \mathbb{E}[\mathbf{g}]_t / (1-\beta_1), \\
\mathbf{v}_t &= \mathbb{E}[\mathbf{g}^2]_t / (1-\beta_2), \\
\theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\mathbf{v}_t + \varepsilon}} \odot \mathbf{m}_t.
\end{aligned}$$

The $\mathbf{m}_t$ and $\mathbf{v}_t$ terms correct for an initial bias towards zero. Typical values are $\beta_1 \approx 0.9$, $\beta_2 \approx 0.999$ and $\varepsilon \approx 10^{-7}$.

5. Weight regularisation, dropout and early stopping

Deep learning models are typically very over-parameterised, often with millions of parameters over many layers in the model. The fitting power enabled by all these parameters makes them **universal approximators** (see e.g. Cybenko [2] for the large width case, or Lu et al. [5] for the large depth case), but on the other hand **the risk of overfitting is a major problem**. When training neural networks, it is important to regularise them to prevent overfitting. There are several forms of regularisation to prevent overfitting, but in this section we will look at three in particular: weight regularisation, dropout and early stopping. Of course, all of these regularisation techniques mentioned can be used together in deep learning models (and they often are).

[1] http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

5.1. L_2 and L_1 regularisation. Recall that for linear regression (Chapter 1), a typical regularisation to add a penalty term containing the squared norm of the parameters to the loss, to discourage the parameters from growing too large. We can write schematically a regularised loss of this type, when the parameters are neural networks' weights $\mathbf{w}$, as:

$$L(\mathbf{w}, \alpha) = L_0(\mathbf{w}) + \alpha_2 \sum_i w_i^2,$$

where L_0 is the unconstrained loss function, and α_2 is a regularisation hyperparameter. This is the L_2 regularisation, in neural networks often referred to as weight decay.

> *Quick Aside.* Note that L_2 regularisation and weight decay, in general, are technically not the same. Weight decay (Hanson & Pratt [4]) is defined as a modification to the update rule, rather than to the loss function itself:
>
> $$\boldsymbol{\theta}_{t+1} \leftarrow (1 - \lambda)\boldsymbol{\theta}_t - \eta \eth_t,$$
>
> where λ and η are hyperparameters, and $\eth_t$ is the t-th batch update. In the case of stochastic gradient descent, the update $\eth_t = \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t; \mathcal{S}_m)$ and the two formulations are equivalent. However, this is not the case for all gradient-based optimisers commonly used in deep learning.

An alternative weight regularisation is the L_1 regularisation, in which the sum of absolute values of the weights are added to the loss term:

$$L(\mathbf{w}, \alpha) = L_0(\mathbf{w}) + \alpha_1 \sum_i |w_i|.$$

This form of regularisation, akin to LASSO, encourages sparsity in the weights. Both L_1 and L_2 regularisation discourage the weights from growing too large, restricting fitting capacity of the network while keeping it away from overfitting, like one has seen for Ridge and Lasso regression (Chapter 1). It is also possible to add a weighted combination of both L_2 and L_1 regularisation to the loss function, similarly to an elastic net.

5.2. Dropout. Dropout was introduced in 2014 by Srivastava et al. [18] (see folder "additional reading") as a regularisation technique for neural networks, that also has the effect of modifying the behaviour of neurons within a network.

The method of dropout is to randomly 'zero out' neurons (or equivalently, weight connections) in the network during training according to a Bernoulli mask whose values are independently sampled at every iteration.

Take $\mathbf{W}^{(l)} \in \mathbb{R}^{n_{l+1} \times n_l}$, the a weight matrix mapping neurons in layer l to layer $l + 1$:

$$\mathbf{h}^{(l+1)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)}\right).$$

We can view dropout as randomly replacing each column of $\mathbf{W}^{(l)}$ with zeros with a certain probability, known as the *dropout rate*. We can write this as applying a Bernoulli mask:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} \cdot \mathrm{diag}(\mathbf{z}^l)$$

where, for layer l, $\mathbf{z}^l$ is a vector of independent n_l Bernoulli random variables. Each of them has probability p_l of being 1:

$$z_j^l \sim \mathrm{Bernoulli}(p_l), \qquad j = 1, ..., n_l,$$

with $\mathbf{W}^{(l)} \in \mathbb{R}^{n_{l+1} \times n_l}$ and $l = 1, \ldots, N_L$. hence, p_l is the probability of keeping a neuron, and the dropout rate is $1 - p_l$. Figures 7.9 and 7.10 illustrate the effect of dropout on a neural network.

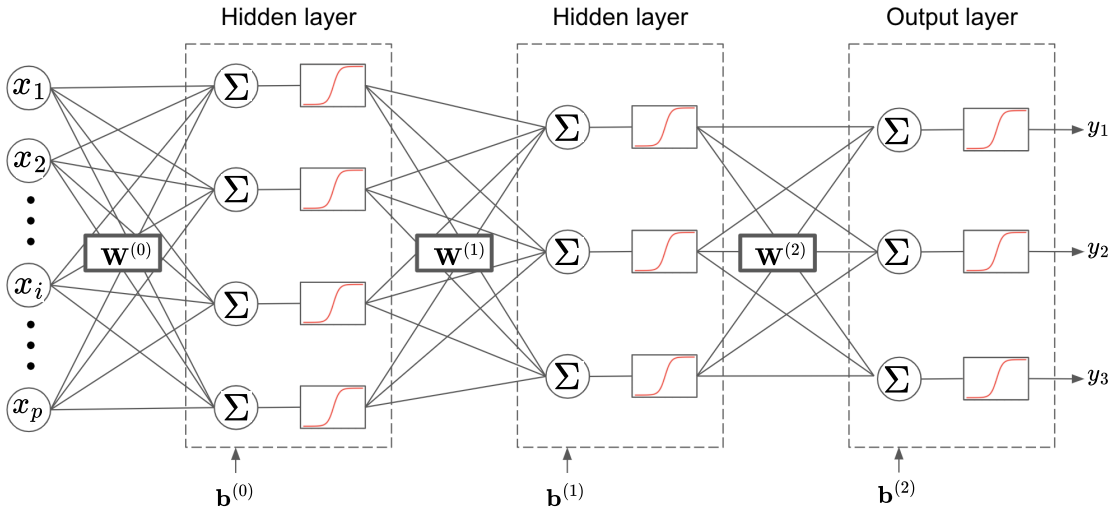


Figure 7.9. Neural network without dropout.

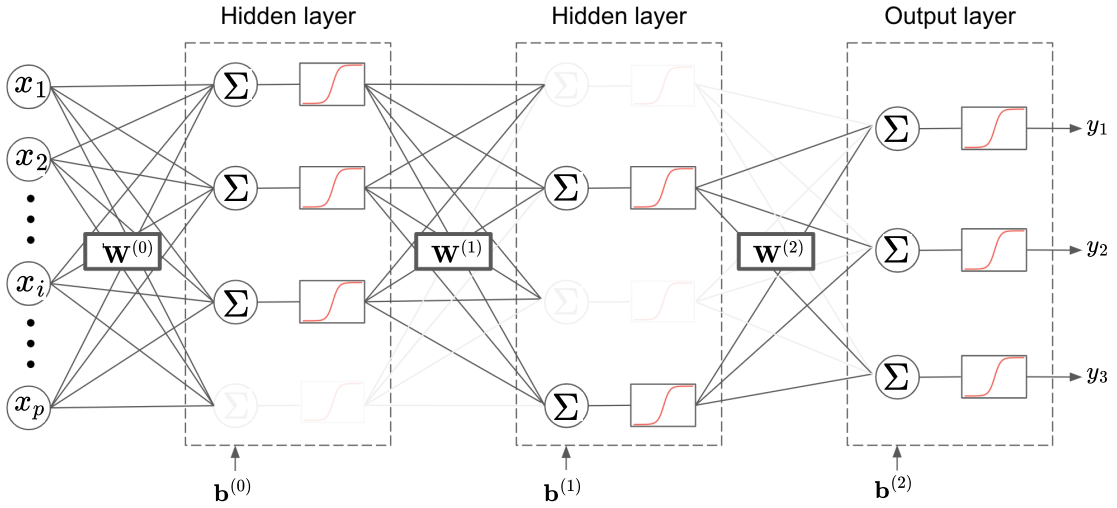
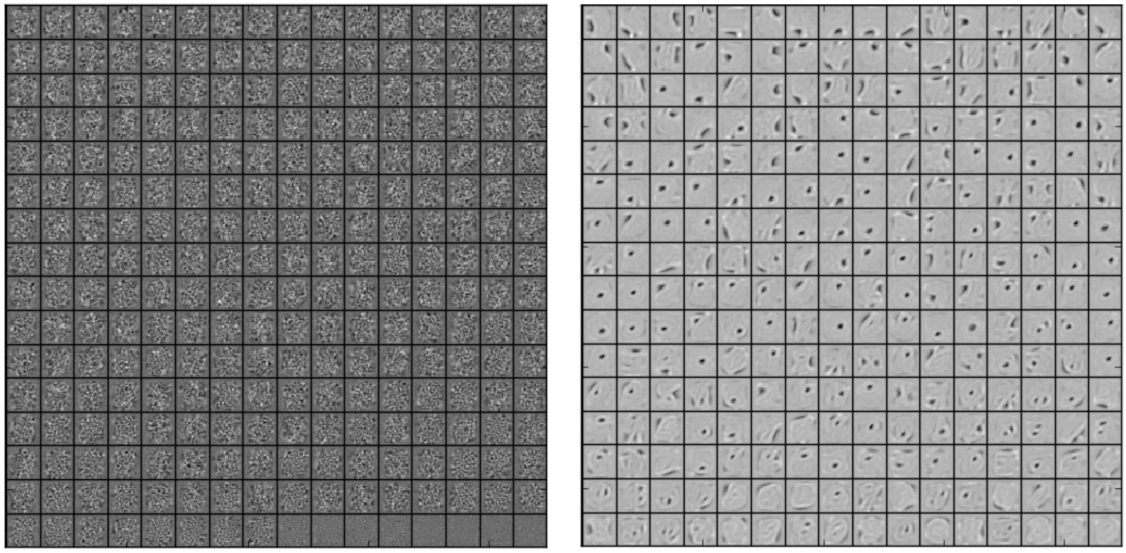


Figure 7.10. Neural network with dropout.

By randomly dropping out neurons in the network, one obvious effect is that the capacity of the model is reduced, and so there is a regularisation effect. Each randomly sampled Bernoulli mask defines a new 'sub-network' that is smaller than the original.

In addition, **a key motivation of dropout is that it prevents neurons from co-adapting too much**. Any neuron in the network is no longer able to depend on any other specific neurons being present, and so each neuron learns features that are more robust, and generalise better.

In the Figure 7.11 (taken from the original paper [18]) we see features that are learned on the MNIST data set for a model trained without dropout (left) and one trained with dropout (right). We see that the dropout model learns features that are much less noisy and more meaningful (it is detecting edges, textures, spots etc.) and help the model to generalise better. The non-dropout model's features suggest a large degree of co-adaptation, where the neurons depend on the specific combination of features in order to make good predictions on the training data.



(a) Without dropout (b) Dropout with $p = 0.5$.

Figure 7.11. Learned features in a neural network trained without dropout (left) and with dropout (right). From Srivastava et al. 2014 [18].

Typically, dropout is applied only in the training phase. When making predictions, all weight connections $\mathbf{W}^{(l)}$ are restored, but re-scaled by a factor of p_l, to take account for the fact that fewer connections were present at training.

5.3. Early stopping. In neural network training, one epoch is a set of iterations that allows for one complete pass through the all the mini-batches of the training dataset, with one iteration per mini-batch. In practice, it is difficult to set a good number of epochs to train for ahead of time. If the training is quick, one can experiment with different values, but in many cases training can take hours or days (or even longer!), so this is not an option.

Recall that deep learning models are usually vastly overparameterised, and have the capacity to drastically overfit. A simple but effective method is to simply stop the training before the model starts to overfit. The resulting picture is similar to the balance between model capacity and generalisation, see Figure 7.12.

With early stopping, the aim is to stop the training when the validation error is at a minimum. This means that the model needs to be regularly evaluated on a held-out validation set (that is not used for training), and the optimisation routine is terminated when the validation error starts to rise. Validation is normally performed once per epoch in the training run.

In practice, the validation error measurements will be noisy, and so it is not a reliable measure to simply detect when the validation error increases and immediately stop the training. What is usually done is to **periodically save model checkpoints (say once per epoch), and set a *patience* threshold**, to specify a maximum number of validation runs that are allowed where the validation error does not improve upon the best score so far. If this patience threshold is reached, the training is terminated.

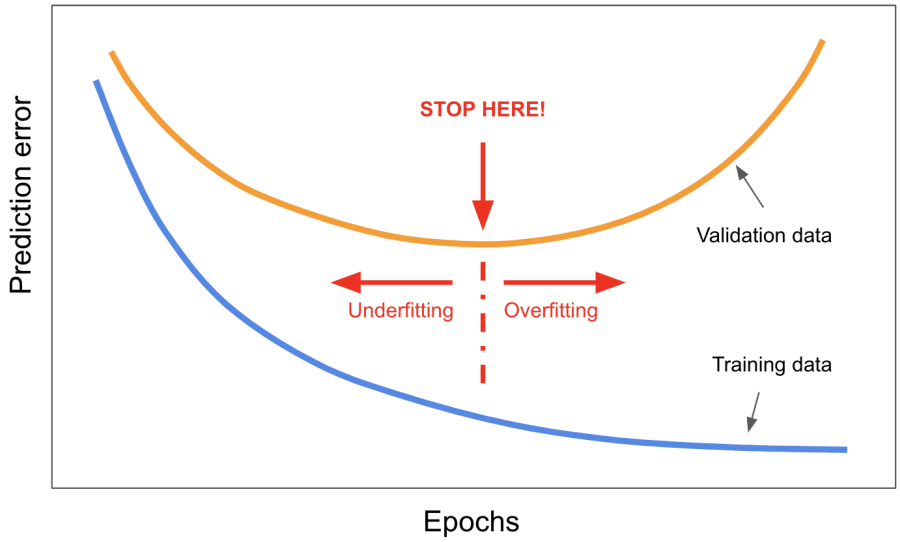


Figure 7.12. Prediction error vs number of training epochs.

References

[1] Clevert, D.-A., Unterthiner, T., & Hochreiter, S. (2016), "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)", in "4th International Conference on Learning Representations, ICLR 2016", San Juan, Puerto Rico, May 2-4, 2016.

[2] Cybenko, G. (1989) "Approximations by superpositions of sigmoidal functions", Mathematics of Control, Signals, and Systems, **2** (4), 303–314.

[3] Duchi, J., Hazan, E., & Singer, Y. (2011), "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization", *Journal of Machine Learning Research*, **12**, 2121–2159.

[4] Hanson, S. J. & Pratt, L. Y. (1988) "Comparing biases for minimal network construction with back-propagation", in *Proceedings of the 1st International Conference on Neural Information Processing Systems*, 177–185.

-

[5] Kingma, D. P. & Ba, J. L. (2015), "Adam: a Method for Stochastic Optimization", International Conference on Learning Representations, 1–13.

[6] Klambauer, G., Unterthiner, T., Mayr, A., & Hochreiter, S. (2017), "Self-Normalizing Neural Networks", *Neural Information Processing Systems (NIPS)*, 971-980.

[7] LeCun, Y., Bengio, Y., Hinton, G. (2015) "Deep Learning", *Nature*, 521(7553), 436-444.

[8] Lu, Z., Pu, H., Wang, F. Hu, Z., & Wang, L. (2017) "The Expressive Power of Neural Networks: A View from the Width", Advances in Neural Information Processing Systems 30. Curran Associates, Inc., 6231–6239.

[9] McCulloch, W. & Pitts, W. (1943), "A Logical Calculus of Ideas Immanent in Nervous Activity", Bulletin of Mathematical Biophysics, **5**, 127-147.

[10] Nesterov, Y. (1983), "A method for unconstrained convex minimization problem with the rate of convergence o(1/k2)", Doklady ANSSSR (translated as Soviet. Math. Docl.), **269**, 543–547.

[11] Pennington, J., Socher, R. & Manning, C. D. (2014), "Glove: Global vectors for word representation", in *Proceedings of Empirical Methods in Natural Language Processing (EMNLP)*.

[12] Qian, N. (1999), "On the momentum term in gradient descent learning algorithms", Neural Networks: The Official Journal of the International Neural Network Society, **12** (1), 145–151.

[13] Ramachandran, P., Zoph, B. & Le, Q. V. (2018) "Searching for Activation Functions", arXiv preprint, abs/1710.05941.

[14] Rosenblatt, F. (1958), "The Perceptron: A Probabilistic Model for Information Storage and Organization in The Brain", Psychological Review, 65-386.

[15] Rumelhart, D. E., McClelland, J. L. and the PDP Research Group (1986a), "Parallel Distributed Processing: Explorations in the Microstructure of Cognition", MIT Press, Cambridge.

[16] Rumelhart, D. E., Hinton, G., & Williams, R. (1986b), "Learning representations by back-propagating errors", Nature, **323**, 533-536.

[17] Rumelhart, D. E., Hinton, G., and Williams, R. (1986c), "Learning Internal Representations by Error Propagation", in Rumelhart, D. E.; McClelland, J. L. (eds.), Parallel Distributed Processing : Explorations in the Microstructure of Cognition. Volume 1: Foundations, Cambridge, MIT Press.

[18] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014), "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", Journal of Machine Learning Research, **15**, 1929-1958.

[19] Werbos, P. J. (1994), "The Roots of Backpropagation: From Ordered Derivatives to Neural Networks and Political Forecasting", New York:, John Wiley & Sons.